Attention Mechanism in Neural Networks

Limitations of RNNs and LSTMs

Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs) have been commonly used for sequence data. However, they have some limitations:

1. Long-range dependencies: RNNs and LSTMs struggle to remember information from earlier parts of the sequence when processing long sequences, leading to a problem called "vanishing gradients."

2. Sequential Processing: Both RNNs and LSTMs process sequences one step at a time, which makes it difficult to parallelize and increases the training time.

3. Overfitting: When the sequence is too long or complex, the model may become prone to overfitting, and handling dependencies between distant words becomes problematic.

Introduction to the Attention Mechanism

The attention mechanism overcomes these limitations by allowing the model to dynamically attend to different parts of the input sequence. This allows for more efficient and effective learning of long-range dependencies. Below, we will break down the attention mechanism into its core concepts.

Mathematical Foundation

Given an input sequence $X = \{x_1, x_2, \ldots, x_n\}$, where each x_i represents an embedding or feature vector, **the goal is to calculate a weighted representation of these inputs.**

Steps of Attention

1. **Scoring:** For each input x_i, calculate a score e_i representing its importance relative to a query vector q:

$$e_i = f(q, x_i),$$

where $f(q, x_i)$ can be:

- **Dot product:** $e_i = q^T x_i$,
- **Additive (MLP):** $e_i = \text{ReLU}(W[q; x_i])$.

2. **Normalization:** Convert scores $\{e_1, e_2, \ldots, e_n\}$ into probabilities using the softmax function:

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{n} \exp(e_j)},$$

where α_i represents the attention weight for x_i.

3. **Weighted Summation:** Compute the **context vector** c, a weighted sum of the input vectors:

$$c = \sum_{i=1}^{n} \alpha_i x_i.$$

Query, Key, and Value

The attention mechanism operates by focusing on specific parts of the input sequence using three main components:

- Query (Q): What we are looking for (the focus).
- Key (K): What we are comparing against (all possible pieces of information).
- Value (V): The actual information we want to extract (the answer).

The model computes a score that measures **how relevant each key is to the given query**. Then, it uses a softmax function to assign attention weights, determining how much attention should be paid to each key.

The core formula of attention is based on computing weighted averages of the values:

$$\text{Attention}(Q, K, V) = \sum_i \alpha_i V_i$$

where:

$$\alpha_i = \frac{\exp(\text{Score}(Q, K_i))}{\sum_j \exp(\text{Score}(Q, K_j))}$$

and

$$\text{Score}(Q, K_i) = Q \cdot K_i$$

is the dot product between the query and each key. This gives us a mechanism to assign different importance levels to different values based on how well their corresponding keys match the query.

Sometimes, we use the following formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

1. **Similarity Scores**:

$$QK^\top \in \mathbb{R}^{n_q \times n_k}$$

- Each row i represents the similarity of query i (row of Q) with all keys (rows of K). - Each entry (i, j) measures how similar query i is to key j.

2. **Scaling**:

$$\frac{QK^\top}{\sqrt{d_k}}$$

- Scaling ensures numerical stability, especially when d_k, the feature dimension of keys/queries, is large.

3. **Softmax Normalization**:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{n_q \times n_k}.$$

- Each row of A contains the attention weights for a single query over all keys. - $A[i, j]$ is the attention weight for query i and key j. These weights are normalized such that:

$$\sum_{j=1}^{n_k} A[i, j] = 1, \quad \forall i.$$

Weighted Average View

The attention output is computed as a **weighted average** of the values:

$$\text{Attention}(Q, K, V) = AV.$$

Here: - $A[i, :]$ represents the attention weights for query i. - The weighted sum for query i is:

$$\text{Attention Output}[i, :] = \sum_{j=1}^{n_k} A[i, j] \cdot V[j, :],$$

where: - $A[i, j]$ is the weight of key j for query i, - $V[j, :]$ is the value vector associated with key j.

Thus, **each row of the attention output is a weighted average of the rows of V, with weights given by the corresponding row of A.**

Row and Column Perspective

- **Rows of Q**: Each row corresponds to a query vector and determines the attention weights in a row of A.
- **Rows of K**: Each row corresponds to a key vector, contributing to the columns of A, which encode the relationships between queries and a specific key.
- **Rows of V**: Each row corresponds to a value vector, combined using attention weights to produce the final attention output.
- **Columns of Q, K, V**: Represent feature dimensions for queries, keys, and values, determining the size of the output feature vectors.

Intuition

The attention weights A act as coefficients in a weighted sum:

$$\text{Attention Output}[i, :] = \sum_{j=1}^{n_k} A[i, j] \cdot V[j, :].$$

This mechanism dynamically adjusts the importance of each key-value pair for every query, allowing the model to focus on the most relevant information.

Visualizing Attention Weights

Attention weights can be visualized using a heatmap:

- **Rows**: Represent different queries.
- **Columns**: Represent different keys.
- **Cells**: Represent the attention weight $A[i, j]$, indicating the importance of key j for query i.

Self-Attention in Transformers

In self-attention, the model compares each word (or token) in the sequence to every other word in the sequence, including itself. This allows the model to weigh how important each word is to others at each step. Mathematically, self-attention uses the same formulas as general attention but applies them across all elements of the sequence.

Let X represent the sequence of inputs, and the model computes:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

Where W^Q, W^K, W^V are learnable weight matrices. Then the attention is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, d_k is the dimension of the key vectors, and dividing by $\sqrt{d_k}$ helps control the magnitude of the dot products to prevent the values from getting too large.

Why Attention is Powerful

1. Focus on Important Information: The attention mechanism allows the model to directly focus on the most relevant parts of the sequence, which solves the long-range dependency problem of RNNs and LSTMs. 2. Parallelization: In attention-based models like Transformers, the sequence is processed in parallel, making them much faster and more efficient for large datasets.

Example of Attention in Practice

Consider translating the sentence "The cat is small" to French ("Le chat est petit"). When producing the word "petit," the attention mechanism focuses more on the word "small" than on other words in the English sentence.

Using the attention formula: - The query focuses on the word "petit." - The keys are the words in the English sentence. - The values are the representations of the English words.

The model computes the attention scores, assigns probabilities, and uses the weighted sum of the values to translate "small" to "petit."

The attention mechanism is central to modern neural network architectures, particularly Transformers. It provides flexibility in handling long sequences and capturing important dependencies between input elements. Unlike RNNs and LSTMs, attention enables parallel processing and avoids problems like vanishing gradients, making it one of the most significant advancements in neural network design.

Attention Mechanism Applied to a Sentence

The attention mechanism processes the input sequence by assigning a weight to each token based on its relevance to other tokens in the sequence. Let's consider the example sentence: *"I like cats."* The attention mechanism will produce a matrix that represents how much each token attends to other tokens in the sentence.

Step-by-Step Process

1. Tokenization: First, the sentence is tokenized into individual words:

"I", "like", "cats"

In this case, we have 3 tokens.

2. Query, Key, and Value Matrices: Each token is mapped to an embedding vector of dimension d using an embedding layer (e.g., Word2Vec, GloVe, or learned embeddings). For simplicity, assume the embedding dimension is d. Now, we have:

$$Q \in \mathbb{R}^{3\times d}, \quad K \in \mathbb{R}^{3\times d}, \quad V \in \mathbb{R}^{3\times d}$$

where Q, K, and V represent the query, key, and value matrices, respectively, for each token.

3. Dot-Product Attention: The attention scores are computed using the dot-product between the query and key matrices:

$$\text{Scores} = QK^T$$

Since $Q \in \mathbb{R}^{3\times d}$ and $K \in \mathbb{R}^{3\times d}$, the result is an attention score matrix of size 3×3, which looks like this:

$$\text{Attention Scores} = \begin{bmatrix} \text{score}_{11} & \text{score}_{12} & \text{score}_{13} \\ \text{score}_{21} & \text{score}_{22} & \text{score}_{23} \\ \text{score}_{31} & \text{score}_{32} & \text{score}_{33} \end{bmatrix}$$

Each score_{ij} represents the attention score between token i and token j.

4. Softmax: To convert the scores into probabilities, the softmax function is applied row-wise:

$$\alpha_{ij} = \frac{\exp(\text{score}_{ij})}{\sum_{j'} \exp(\text{score}_{ij'})}$$

This results in the attention matrix α, where each row contains values between 0 and 1, and the sum of each row is 1.

5. Weighted Sum: The attention matrix α is used to compute a weighted sum of the value vectors:

$$\text{Attention Output} = \alpha \cdot V$$

This results in a matrix of size $3 \times d$, where each row is a weighted sum of the value vectors.

Example: Attention Matrix for "I like cats"

After applying the softmax function to the attention scores, the attention matrix might look like this (assuming hypothetical values):

$$\alpha = \begin{bmatrix} 0.7 & 0.2 & 0.1 \\ 0.1 & 0.6 & 0.3 \\ 0.2 & 0.3 & 0.5 \end{bmatrix}$$

This matrix represents how much each token attends to other tokens:

- The first row (0.7, 0.2, 0.1) means that "I" attends mostly to itself (0.7), less to "like" (0.2), and even less to "cats" (0.1).
- The second row (0.1, 0.6, 0.3) means that "like" attends more to itself (0.6) and slightly to "cats" (0.3).
- The third row (0.2, 0.3, 0.5) means that "cats" attends mostly to itself (0.5) but also considers "like" (0.3).

Final Attention Output

The final attention matrix α has size 3×3, where each row sums to 1. If the embedding dimension d is, for example, 64, then the final output after computing the weighted sum of the value vectors will be a matrix of size 3×64.

Thus, the attention mechanism produces a context-aware representation of each token by focusing on other relevant tokens in the sentence, captured in the attention matrix.

Multi-Head Attention and Concatenation Example

Consider the sentence `"I like cats"`. Assume we have 2 attention heads and each word in the sentence is represented by a 4-dimensional embedding. After performing self-attention for each head, we obtain the following output embeddings:

Head 1 outputs:

$$\begin{aligned} \text{I} &: [0.1, 0.2, 0.3, 0.4], \\ \text{like} &: [0.2, 0.3, 0.4, 0.5], \\ \text{cats} &: [0.3, 0.4, 0.5, 0.6]. \end{aligned}$$

Head 2 outputs:

$$\begin{aligned} \text{I} &: [0.5, 0.4, 0.3, 0.2], \\ \text{like} &: [0.6, 0.5, 0.4, 0.3], \\ \text{cats} &: [0.7, 0.6, 0.5, 0.4]. \end{aligned}$$

Now, the outputs from both heads are concatenated to form a single 8-dimensional vector for each word.

Concatenated outputs:

$$\begin{aligned} \text{I} &: [0.1, 0.2, 0.3, 0.4, 0.5, 0.4, 0.3, 0.2], \\ \text{like} &: [0.2, 0.3, 0.4, 0.5, 0.6, 0.5, 0.4, 0.3], \\ \text{cats} &: [0.3, 0.4, 0.5, 0.6, 0.7, 0.6, 0.5, 0.4]. \end{aligned}$$

After concatenation, a linear transformation is applied to reduce the dimensionality back to the original size.

Positional Encoding

In Transformer models, positional embeddings (also known as positional encodings) are added to the input embeddings to inject the notion of order into the input sequences since the self-attention mechanism itself does not take into account the sequence's position. These positional encodings are computed using sine and cosine functions of different frequencies.

$$\begin{aligned} PE(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d}}\right), \\ PE(pos, 2i+1) &= \cos\left(\frac{pos}{10000^{2i/d}}\right), \end{aligned}$$

where pos is the word position, i is the dimension index, and d is the embedding size.

For example, assuming an embedding size of 4, the positional encodings for the words in the sentence might look like this:

Positional encodings:

$$\begin{aligned} \text{I} &: [0.0, 1.0, 0.0, 1.0], \\ \text{like} &: [0.84, 0.54, 0.98, 0.29], \\ \text{cats} &: [0.91, 0.41, 0.99, 0.18]. \end{aligned}$$

These positional encodings are added to the word embeddings before the self-attention mechanism is applied.

More specifically, for a given position pos and a dimension i, the positional encoding $PE(pos, i)$ is computed as follows:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Where:

- pos is the position in the sequence (starting from 0),
- i is the dimension of the positional embedding,
- d_{model} is the dimensionality of the model (or the size of the embeddings).

Example Let's compute the positional embeddings for a small sequence with $pos = 0$, $pos = 1$, and $pos = 2$, assuming $d_{\text{model}} = 4$. For this example, we'll calculate the embeddings for the first four dimensions of the model (i.e., $i = 0, 1, 2, 3$).

1. For $pos = 0$:

$$PE(0, 0) = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0$$
$$PE(0, 1) = \cos\left(\frac{0}{10000^{0/4}}\right) = \cos(0) = 1$$
$$PE(0, 2) = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0$$
$$PE(0, 3) = \cos\left(\frac{0}{10000^{2/4}}\right) = \cos(0) = 1$$

Thus, the positional encoding for $pos = 0$ is:

$$PE(0) = [0, 1, 0, 1]$$

2. For $pos = 1$:

$$PE(1, 0) = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1)$$
$$PE(1, 1) = \cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1)$$
$$PE(1, 2) = \sin\left(\frac{1}{10000^{2/4}}\right) = \sin\left(\frac{1}{100}\right)$$

$$PE(1,3) = \cos\left(\frac{1}{10000^{2/4}}\right) = \cos\left(\frac{1}{100}\right)$$

Using approximate values:

$$PE(1,0) \approx \sin(1) \approx 0.8415$$
$$PE(1,1) \approx \cos(1) \approx 0.5403$$
$$PE(1,2) \approx \sin(0.01) \approx 0.01$$
$$PE(1,3) \approx \cos(0.01) \approx 0.99995$$

Thus, the positional encoding for $pos = 1$ is:

$$PE(1) \approx [0.8415, 0.5403, 0.01, 0.99995]$$

3. For $pos = 2$:

$$PE(2,0) = \sin\left(\frac{2}{10000^{0/4}}\right) = \sin(2)$$
$$PE(2,1) = \cos\left(\frac{2}{10000^{0/4}}\right) = \cos(2)$$
$$PE(2,2) = \sin\left(\frac{2}{10000^{2/4}}\right) = \sin\left(\frac{2}{100}\right)$$
$$PE(2,3) = \cos\left(\frac{2}{10000^{2/4}}\right) = \cos\left(\frac{2}{100}\right)$$

Using approximate values:

$$PE(2,0) \approx \sin(2) \approx 0.9093$$
$$PE(2,1) \approx \cos(2) \approx -0.4161$$
$$PE(2,2) \approx \sin(0.02) \approx 0.02$$
$$PE(2,3) \approx \cos(0.02) \approx 0.9998$$

Thus, the positional encoding for $pos = 2$ is:

$$PE(2) \approx [0.9093, -0.4161, 0.02, 0.9998]$$

Summary of Positional Embeddings
For $d_{\text{model}} = 4$, the positional encodings for positions 0, 1, and 2 are approximately:

$$PE(0) = [0, 1, 0, 1]$$
$$PE(1) \approx [0.8415, 0.5403, 0.01, 0.99995]$$
$$PE(2) \approx [0.9093, -0.4161, 0.02, 0.9998]$$

Multi-Head Attention with Positional Encoding: Combined Example

The example sentence is `"I like cats"`.

Step 1: Word Embeddings Each word in the sentence is first converted into a fixed-size vector representation (embedding). For simplicity, assume each word is represented by a 4-dimensional embedding:

$$\begin{aligned} \text{I} &: [0.1, 0.3, 0.4, 0.6], \\ \text{like} &: [0.5, 0.4, 0.3, 0.2], \\ \text{cats} &: [0.6, 0.7, 0.5, 0.3]. \end{aligned}$$

These embeddings capture some basic meaning of the words, but they lack information about the word order and relationships between words in the sentence.

Step 2: Positional Encoding Since the Transformer does not have inherent knowledge of the word order, positional encoding is added to the word embeddings. The positional encoding is calculated using sine and cosine functions of different frequencies:

$$\begin{aligned} PE(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d}}\right), \\ PE(pos, 2i+1) &= \cos\left(\frac{pos}{10000^{2i/d}}\right), \end{aligned}$$

where pos is the word position, i is the dimension index, and d is the embedding size.

For our example sentence, assuming an embedding size of 4, the positional encodings might look like this:

Positional encodings:

$$\begin{aligned} \text{I (pos} = 1) &: [0.0, 1.0, 0.0, 1.0], \\ \text{like (pos} = 2) &: [0.84, 0.54, 0.98, 0.29], \\ \text{cats (pos} = 3) &: [0.91, 0.41, 0.99, 0.18]. \end{aligned}$$

These positional encodings are added to the word embeddings to include information about the word order:

Word embeddings + Positional encodings:

$$\begin{aligned} \text{I} &: [0.1, 1.3, 0.4, 1.6], \\ \text{like} &: [1.34, 0.94, 1.28, 0.49], \\ \text{cats} &: [1.51, 1.11, 1.49, 0.48]. \end{aligned}$$

Step 3: Self-Attention Next, the self-attention mechanism is applied. For simplicity, let's assume we have 2 attention heads. Each head produces a different set of learned weights, allowing the model to focus on different aspects of the sentence.

Head 1 outputs:

$$\begin{aligned} \text{I} &: [0.1, 0.2, 0.3, 0.4], \\ \text{like} &: [0.2, 0.3, 0.4, 0.5], \\ \text{cats} &: [0.3, 0.4, 0.5, 0.6]. \end{aligned}$$

Head 2 outputs:

$$\begin{aligned} \text{I} &: [0.5, 0.4, 0.3, 0.2], \\ \text{like} &: [0.6, 0.5, 0.4, 0.3], \\ \text{cats} &: [0.7, 0.6, 0.5, 0.4]. \end{aligned}$$

Step 4: Concatenation The outputs from both attention heads are concatenated to form a single 8-dimensional vector for each word:

Concatenated outputs:

$$\begin{aligned} \text{I} &: [0.1, 0.2, 0.3, 0.4, 0.5, 0.4, 0.3, 0.2], \\ \text{like} &: [0.2, 0.3, 0.4, 0.5, 0.6, 0.5, 0.4, 0.3], \\ \text{cats} &: [0.3, 0.4, 0.5, 0.6, 0.7, 0.6, 0.5, 0.4]. \end{aligned}$$

After this concatenation, a linear transformation is applied to reduce the dimensionality back to the original size (e.g., 4 dimensions).

Step 5: Attention Matrix The attention matrix for a sentence of 3 words (like `"I like cats"`) has dimensions 3×3 since each word attends to every other word in the sentence. This matrix contains the attention weights indicating how much each word attends to the others.

For instance, if the attention matrix is:

$$\begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.3 & 0.5 & 0.2 \\ 0.2 & 0.4 & 0.4 \end{bmatrix}$$

it means that the first word "I" focuses mainly on itself (0.8), the word "like" attends equally to itself and the word "cats," and "cats" splits its attention between the other two words.

Encoder Process:

The encoder process in a Transformer model can be summarized in the following steps:

1. **Positional Embeddings:** First, we compute the positional embeddings and add them to the input word embeddings. This is crucial because it allows the model to encode information about the order of words in the sequence, which is not inherently captured by the word embeddings alone.

2. **Multi-head Attention Setup:** Next, we determine the number of heads for the multi-head attention mechanism. For each head, we compute the query, key, and value matrices by multiplying the input embeddings with learned weights. These matrices will be used in the attention mechanism to capture different relationships between words in the input sequence.

3. **Multi-head Attention Process:** For each head, the attention mechanism works as follows: the query and key matrices are multiplied (taking the dot product) to compute attention scores. These scores are scaled, passed through a softmax function, and then used to weight the value matrices. This generates several attention outputs, one for each head, each capturing different aspects of the input.

4. **Concatenation and Final Layers:** The outputs from all attention heads are concatenated. The concatenated output is then passed through a small feed-forward neural network consisting of two fully connected layers with ReLU activations. This helps further process and refine the attention outputs, completing the core operations of the encoder block.

Complete Explanation for a Translation Task using a Transformer with One Attention Head

Task Setup

- **Input**: Source sentence to be translated (e.g., "this is cat").
- **Target**: Translated sentence in the target language (e.g., "le chat dort").
- **Model**: Transformer with a single head for self-attention and cross-attention.

We define the following dimensions:

- L: Length of the source sentence.
- M: Length of the target sentence.
- d: Embedding dimension.
- V: Size of the target vocabulary.

Step-by-Step Explanation

1. Word Embedding and Positional Encoding (in Encoder)

- The input sentence "this is cat" is embedded into a matrix X of shape (L, d).
- Positional encodings are added to the input embeddings:

$$X' = X + PE_{\text{enc}},$$

where PE_{enc} is a matrix of positional encodings of shape (L, d).

2. Encoder: Self-Attention with One Head

- Compute the queries (Q), keys (K), and values (V) using learned weight matrices of shape (d, d):

$$Q = X'W_Q, \quad K = X'W_K, \quad V = X'W_V.$$

- Calculate the attention scores using scaled dot-product attention:

$$Z_{\text{encoder}} = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V.$$

3. Residual Connection and Layer Normalization (in Encoder)

- Add a residual connection and apply layer normalization:

$$Z'_{\text{encoder}} = \text{LayerNorm}(Z_{\text{encoder}} + X').$$

4. Feedforward Neural Network in the Encoder

- Pass Z'_{encoder} through a feedforward neural network:

$$\text{Feedforward}(X) = \text{ReLU}(XW_1 + b_1)W_2 + b_2.$$

- Apply a residual connection and layer normalization:

$$Z''_{\text{encoder}} = \text{LayerNorm}(\text{Feedforward}(Z'_{\text{encoder}}) + Z'_{\text{encoder}}).$$

5. Word Embedding and Positional Encoding (in Decoder)

- The target sentence "le chat dort" is embedded into a matrix Y of shape (M, d).
- Positional encodings are added to the decoder's input embeddings:

$$Y' = Y + PE_{\text{dec}},$$

where PE_{dec} is a matrix of positional encodings of shape (M, d).

6. Decoder: Self-Attention with One Head

- Compute the queries (Q_{dec}), keys (K_{dec}), and values (V_{dec}) using weight matrices of shape (d, d):

$$Q_{\text{dec}} = Y'W_Q^{\text{dec}}, \quad K_{\text{dec}} = Y'W_K^{\text{dec}}, \quad V_{\text{dec}} = Y'W_V^{\text{dec}}.$$

- Calculate the attention scores:

$$Z_{\text{dec}} = \text{softmax}\left(\frac{Q_{\text{dec}}K_{\text{dec}}^T}{\sqrt{d}}\right)V_{\text{dec}}.$$

- Add a residual connection and apply layer normalization:

$$Z'_{\text{dec}} = \text{LayerNorm}(Z_{\text{dec}} + Y').$$

7. Cross-Attention in the Decoder

- Compute the queries (Q_{cross}), keys (K_{cross}), and values (V_{cross}) from the decoder's self-attention output and the encoder's output:

$$Q_{\text{cross}} = Z'_{\text{dec}}W_Q^{\text{cross}}, \quad K_{\text{cross}} = Z''_{\text{encoder}}W_K^{\text{cross}}, \quad V_{\text{cross}} = Z''_{\text{encoder}}W_V^{\text{cross}}.$$

- Calculate the cross-attention scores:

$$Z_{\text{cross}} = \text{softmax}\left(\frac{Q_{\text{cross}}K_{\text{cross}}^T}{\sqrt{d}}\right)V_{\text{cross}}.$$

- Add a residual connection and apply layer normalization:

$$Z_{\text{final}} = \text{LayerNorm}(Z_{\text{cross}} + Z'_{\text{dec}}).$$

8. Output Layer and Loss Calculation

- The decoder's output Z_{final} is of shape (M, d).
- Map each row of Z_{final} to a distribution over the target vocabulary of size V by multiplying with a weight matrix W of shape (d, V):

$$\text{logits}_i = Z_{\text{final},i} \cdot W + b.$$

- Apply the softmax function to obtain the predicted probability distribution for each word in the target vocabulary:

$$\hat{y}_i = \text{softmax}(\text{logits}_i).$$

- Compare the predicted probability distribution $\hat{y}_i$ with the ground-truth target word using cross-entropy loss. For each position i, the loss is given by:

$$\mathcal{L}_i = -\sum_{c=1}^{V} y_{i,c} \log(\hat{y}_{i,c}),$$

  where $y_{i,c}$ is the one-hot encoded ground-truth word at position i.

- The total loss for the entire sentence is:

$$\mathcal{L} = \sum_{i=1}^{M} \mathcal{L}_i.$$

Summary of Each Step

- **Encoder**: Positional encoding, self-attention, residual connection, layer normalization, feedforward network, residual connection, and layer normalization.
- **Decoder**: Positional encoding, self-attention, residual connection, layer normalization, cross-attention, residual connection, layer normalization, feedforward network, residual connection, and layer normalization.
- **Output**: Mapping each position to a probability distribution over the target vocabulary using softmax and computing the cross-entropy loss.

Training Transformers in Practice

Input to Transformers: Sentence as Input?

Yes, in practice, a sentence is given as input to a Transformer model. However, it is not passed in raw text form but undergoes several preprocessing steps. These steps include:

- **Tokenization**: The input sentence is tokenized into smaller units (words, subwords, or characters) depending on the tokenizer (e.g., WordPiece or BPE).
- **Embedding**: Each token is converted into a numerical representation via an embedding layer.
- **Positional Encoding**: Since Transformers process tokens in parallel, positional encodings are added to retain sequence information.
- **Special Tokens**: Tokens like `[CLS]` (classification token) or `[SEP]` (separator token) are added as required by the specific task.

For instance, the input sentence "Transformers are powerful" might be tokenized and represented as:

`[CLS, Transformers, are, powerful, [SEP]]`.

The final representation is a sequence of token embeddings combined with positional encodings, which is passed to the Transformer model.

How is Training Done in Practice?

Training a Transformer involves several steps:

1. **Forward Pass**:
   - The input embeddings are passed through the Transformer layers.
   - Self-attention layers compute relationships between tokens, generating contextual embeddings.
   - Output embeddings from the final layer are used for the specific task (e.g., classification, translation).
2. **Loss Computation**:
   - A loss function (e.g., cross-entropy) computes the error between predictions and ground truth.
3. **Backward Pass**:
   - Gradients of the loss with respect to model parameters are computed via backpropagation.
4. **Parameter Update**:
   - Optimizers like AdamW update model parameters.
   - Learning rate schedules with warm-up and decay phases are often used.
5. **Evaluation**:
   - Validation metrics (e.g., accuracy or BLEU) are monitored.
   - Early stopping can be applied if validation performance stagnates.

Practical Considerations for Training

Several practical considerations ensure efficient and effective training of Transformer models:

- **Memory Efficiency**:
  - Techniques like gradient checkpointing and mixed-precision training reduce memory usage.
- **Batching**:
  - Sequences are padded to the same length in a batch for parallel processing.
  - Larger batch sizes can be simulated using gradient accumulation.
- **Regularization**:
  - Dropout layers prevent overfitting.
  - Label smoothing mitigates overconfidence in predictions.
- **Optimization Stability**:
  - Learning rate schedules ensure stable training.
  - Adaptive optimizers (e.g., AdamW) adjust learning rates dynamically.
- **Monitoring and Early Stopping**:
  - Validation loss or metrics are tracked to detect overfitting.

– Early stopping halts training when improvement ceases.

In deep learning, particularly in transformer architectures, the feedforward and attention components play critical roles. These components are structured mathematically to ensure modularity and compatibility with the overall network design. In this note, we explain why feedforward and attention components can be expressed in matrix form and provide details of the feedforward layer following attention.

Feedforward Components in Neural Networks

In a neural network, the feedforward component operates as a transformation of the input through a weight matrix and bias addition followed by a non-linear activation. Consider a feedforward layer where:

- The current layer has n nodes, each node containing a value z_i for $i = 1, \ldots, n$.
- The next layer consists of m nodes.

The operation can be written as:

$$z_j^{\text{next}} = f\left(\sum_{i=1}^{n} W_{ji} z_i + b_j\right), \quad \text{for } j = 1, \ldots, m,$$

where:

- W_{ji} represents the weight from node i in the current layer to node j in the next layer.
- b_j is the bias associated with the j-th node in the next layer.
- $f(\cdot)$ is the activation function applied element-wise.

This can be expressed in matrix form as:

$$Z^{\text{next}} = f(ZW^T + b),$$

where:

- $Z \in \mathbb{R}^{1\times n}$ is the row vector of current layer activations.
- $W \in \mathbb{R}^{m\times n}$ is the weight matrix.
- $b \in \mathbb{R}^{1\times m}$ is the bias row vector, broadcasted across the output.
- $Z^{\text{next}} \in \mathbb{R}^{1\times m}$ is the output vector for the next layer.

Feedforward Layer After Attention

The feedforward layer applied after attention involves a linear transformation followed by a non-linear activation. The operation can be written as:

$$Z^{\text{output}} = f(Z^{\text{attention}} W_O + \mathbf{1}_n b^T),$$

where:

- $Z^{\text{attention}} \in \mathbb{R}^{n\times d_k}$ is the attention output.
- $W_O \in \mathbb{R}^{d_k \times d_{\text{model}}}$ is the weight matrix for the feedforward layer.
- $b \in \mathbb{R}^{d_{\text{model}}}$ is the bias vector.
- $\mathbf{1}_n \in \mathbb{R}^{n\times 1}$ is a column vector of ones for broadcasting b.
- $f(\cdot)$ is a non-linear activation function (e.g., ReLU, GELU).

Conclusion

Both feedforward and self-attention components can be expressed in a structured matrix form, ensuring computational efficiency and modularity in transformer architectures. The self-attention mechanism builds token-wise contextual relationships, while the feedforward layer processes these relationships to enhance task-specific representation.

Feedforward Layer Representation

A feedforward layer in a neural network is designed to transform the input through a linear transformation followed by a non-linear activation function. Given an input matrix $Z \in \mathbb{R}^{n \times d_{\text{model}}}$, where n is the number of tokens and d_{model} is the feature dimension, the feedforward operation is defined as:

$$Z^{\text{output}} = f(ZW_O + \mathbf{1}_n b^T),$$

where:

- $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{out}}}$ is the weight matrix.
- $b \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector.
- $\mathbf{1}_n \in \mathbb{R}^{n \times 1}$ is a column vector of ones for broadcasting b across tokens.
- $f(\cdot)$ is a non-linear activation function (e.g., ReLU or GELU).
- $Z^{\text{output}} \in \mathbb{R}^{n \times d_{\text{out}}}$ is the output of the feedforward layer.

Why This Representation Works

1. Linear Transformation: The matrix multiplication ZW_O applies the same transformation to all tokens. Each token vector Z_i (a row of Z) is independently mapped to a new space of dimension d_{out}.

2. Bias Addition: The bias vector b is added identically to all tokens, ensuring flexibility in the transformation.

3. Non-linear Activation: The activation function $f(\cdot)$ introduces non-linearity, allowing the network to model complex relationships in the data.

4. Matrix Form: Using matrices ensures computational efficiency and alignment with the parallel processing capabilities of modern hardware (e.g., GPUs/TPUs).

Attention Component Representation

The self-attention mechanism computes attention scores and outputs a weighted sum of the input values. The operation is represented as:

$$Z^{\text{attention}} = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where:

- $Q, K, V \in \mathbb{R}^{n \times d_k}$ are the query, key, and value matrices, respectively, derived from the input Z using learnable weights.
- $\frac{1}{\sqrt{d_k}}$ scales the dot-product for numerical stability.
- Softmax$(\cdot)$ computes the attention weights across the sequence.
- $Z^{\text{attention}} \in \mathbb{R}^{n \times d_k}$ is the output of the attention mechanism.

Why This Representation Works

1. Similarity-Based Weights: The dot-product QK^T measures the similarity between tokens, capturing contextual relationships.

2. Normalization: The softmax function ensures that attention weights are non-negative and sum to 1, maintaining interpretability.

3. Weighted Summation: The product Softmax $\left(\frac{QK^T}{\sqrt{d_k}}\right) V$ computes a weighted average of the values V, contextualized by the attention weights.

Feedforward Layer After Attention

Following the attention layer, a feedforward layer processes the output $Z^{\text{attention}}$. The operation is:

$$Z^{\text{output}} = f(Z^{\text{attention}} W_O + \mathbf{1}_n b^T),$$

where $Z^{\text{attention}} \in \mathbb{R}^{n \times d_k}$, $W_O \in \mathbb{R}^{d_k \times d_{\text{out}}}$, and $b \in \mathbb{R}^{d_{\text{out}}}$.

Practical Implications

1. Dimensionality Alignment: The feedforward layer ensures that the dimensionality of the model remains consistent for subsequent layers.

2. Task-Specific Transformation: By applying a feedforward transformation after attention, the model captures both contextual and hierarchical relationships in the data.

3. Scalability: Using a matrix-based formulation allows for efficient computation, even with large sequences and high-dimensional embeddings.

Conclusion

The representation of feedforward and attention components in matrix form ensures compatibility with the modular structure of transformer architectures. These representations are computationally efficient, scalable, and aligned with the theoretical framework of deep learning models.

Attention Mechanism

The *attention mechanism* is a powerful concept in machine learning that enables models to focus on the most relevant parts of the input when performing tasks. It is inspired by the way humans selectively concentrate on specific parts of visual or textual information. Attention is widely used in applications such as machine translation, image captioning, and recommendation systems.

Key Idea

The attention mechanism assigns different weights (or importance scores) to various parts of the input, guiding the model to focus on the most important information while downplaying irrelevant parts. This allows the model to make better predictions by leveraging the most relevant input features.

Types of Attention

- **Self-Attention:** Every input element attends to all elements in the sequence, including itself. The formula for scaled dot-product self-attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

  where Q, K, V are the query, key, and value matrices, and d_k is the dimensionality of the key vectors.

- **Multi-Head Attention:** Extends self-attention by applying multiple attention heads in parallel to capture different aspects of the input:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O,$$

  where each head is computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V).$$

- **Cross-Attention:** One set of inputs (e.g., a query sequence) attends to another set of inputs (e.g., a context sequence), commonly used in sequence-to-sequence models.

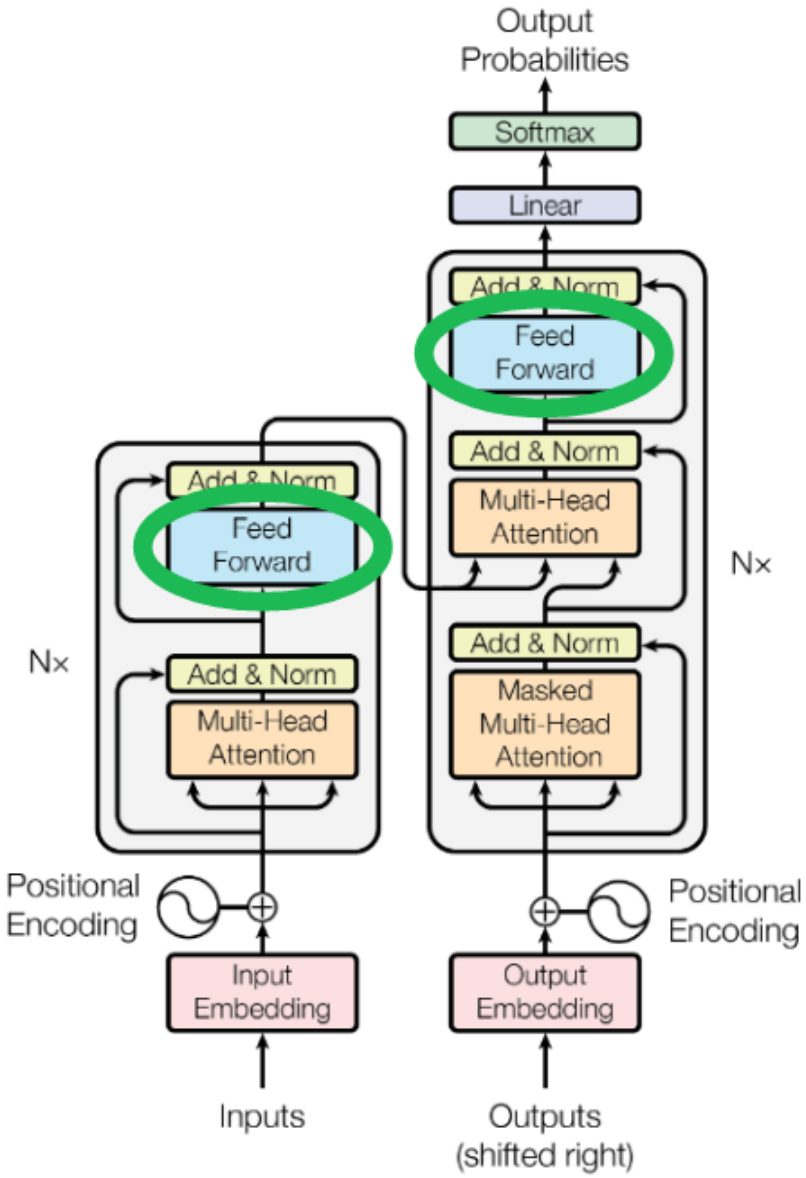
Benefits of Attention

- **Flexibility:** Dynamically focuses on different parts of the input based on the task.
- **Scalability:** Self-attention scales well to large datasets and long sequences.
- **Explainability:** Attention weights provide insights into the importance of input elements.

Applications

- **Natural Language Processing (NLP):** Machine translation, summarization, and question answering.
- **Computer Vision:** Image captioning and object detection.
- **Recommendation Systems:** Context-aware recommendations using user preferences.
- **Time Series Analysis:** Identifying significant patterns in temporal data.

The attention mechanism has revolutionized machine learning, particularly through its use in *Transformers*, forming the backbone of models such as *BERT*, *GPT*, and other state-of-the-art architectures.


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)